

Hybrid-INoT: Isolating Role Labels and Call Structure in Verified Code Generation

Anonymous Author(s)

Submission Id: Pending registration

Abstract

Hybrid-INoT combines planning, implementation, and critique in one model response while leaving correctness verification to an external executable evaluator. We isolate this single-pass core by crossing role labels (neutral versus planner–implementer–reviewer) with call structure (one versus three calls), holding the operations and full supplied task context fixed. A direct solver is the fifth condition. GPT-5.6 Luna with medium reasoning receives 15,000 assignments on 1,000 BigCodeBench tasks with three repeats per condition; 14,961 candidates complete and 985 tasks pass reference controls. Neither role-label quality contrast is significant after Holm adjustment. On complete eligible task pairs, three-call execution lowers success by 4.17 percentage points for neutral instructions and 2.13 points for role-labelled instructions. Its API-equivalent valuation is 2.67 and 2.78 times that of the corresponding single-call workflows. A separate 2,000-block comparison assigns 6,000 fresh single-call and Self-Collaboration workflows across 956 tasks. Self-Collaboration has lower observed-pair success by 2.48–2.68 points and approximately three times the valuation, but substantial infrastructure missingness leaves the full-population quality difference unidentified. The results provide a controlled evaluation of role labels and interaction structure, with explicit boundaries on quality and resource claims. They do not establish that named roles act as independent agents or that one topology dominates all agent workflows.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Automatic programming*.

Keywords

Language model agents, code generation, role prompting, factorial experiments, executable verification

ACM Reference Format:

Anonymous Author(s). 2027. Hybrid-INoT: Isolating Role Labels and Call Structure in Verified Code Generation. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 7 pages.

1 Introduction

Code-generation agents must translate a specification into a program, use supplied code and documentation, and satisfy an executable contract. A workflow may assign these functions to a

planner, implementer, and critic. If the functions are distributed across fresh model calls, however, the complete task context and intermediate responses must be transmitted repeatedly. This creates a concrete architectural question: can the functions be retained inside one response while correctness is checked externally?

Hybrid-INoT addresses this question by combining instructed internal role organization with an external verification layer. The model is asked to analyze requirements and edge cases, implement from that analysis, and review and revise the result. The resulting program is then evaluated by the original benchmark tests in a controlled environment. The model’s textual assertion that a program is correct is not used as the outcome. The experiment studies the core configuration with identity context selection, one final candidate, no compression, and no test-driven reruns.

The distinction between instructed functions and independent agents is central. The labels planner, implementer, and reviewer name parts of a prompt; they do not demonstrate separate latent agents, independent memories, or independent optimization. We therefore compare the labels directly with neutral stage names while keeping operation sentences unchanged. We also compare one and three fresh calls, forwarding every complete previous response. This crossed design separates a label intervention from a call-topology intervention as far as the fixed-context protocol permits.

Our closest prior work includes role ablations and interaction-count studies, including Self-Collaboration and PromptCode-style internal dialogue [2, 9]. The incremental contribution is an auditable, matched component evaluation of the Hybrid-INoT engineering core: crossed role labels and call structure, complete-context accounting, native benchmark evaluation, and explicit missingness. It is not the invention of role collaboration. The main experiment uses 1,000 BigCodeBench tasks, three fresh repeats per condition, and all 14,961 complete candidates. The results provide an empirical boundary for this implementation: additional call boundaries increase measured resources and reduce test success in the tested setting, while role labels alone do not survive the prespecified multiplicity correction.

The paper makes four contributions. First, it gives an operational definition of the core, including its state, stopping rule, extraction boundary, and external evaluator. Second, it states the conditional token balance that motivates a shared response without assuming equal output lengths or a universal long-context crossover. Third, it reports the large factorial study and its task-level quality and resource estimands. Fourth, it places the result beside the earlier 200-task factorial, the 80-task label-by-layout experiment, an INoT prompt adaptation, and development feasibility evidence, preserving their distinct models, environments, and sampling scopes.



This work is licensed under a Creative Commons Attribution International 4.0 License.

2 Related Work and Scope

Self-Collaboration decomposes problem solving into analyst, developer, and tester roles and feeds messages between them [2]. Its Sections 5.2–5.3 already compare role configurations and interaction counts. Our contribution is the joint label-by-topology intervention with unchanged operation sentences, repeated native outcomes and complete context accounting; the SCC comparison evaluates the historical author controller under a disclosed dataset and transport adaptation. INoT motivates compact internal dialogue and role-like reasoning instructions [9]. These works motivate the architectural question but do not make a role label an independently observed agent. Our factorial intervention preserves the stage operations and changes only the labels, whereas the INoT comparison changes the reasoning procedure itself. Results from those interventions are consequently not pooled.

Persona prompting studies likewise distinguish a named persona from a demonstrated performance benefit [3, 13]. Self-Refine and Reflexion use feedback and revision [5, 8]; our primary matrix withholds all executable feedback until generation ends.

Additional-call comparisons can confound call count with computation, repeated context, and exposure of intermediate responses. Work on matched thinking-token budgets emphasizes that these are distinct interventions [10]. We do not impose a hard matched output budget. Instead, we record input, cached input, output, and API-equivalent valuation, and interpret them as properties of the implemented workflows. Agentless and SWE-agent add localization, repository access, or tool interaction [11, 12]; those systems address a broader repository setting than the fixed-context function-generation matrix here. BigCodeBench supplies diverse library-dependent tasks with executable tests [14]. Native evaluation improves auditability while retaining the benchmark’s task-test limitations.

3 Formal Problem and Hybrid-INoT Core

An engineering task is $(x, C_0, \mathcal{V})$, where x is the specification, C_0 is the complete supplied context, and $\mathcal{V}$ is an external verification procedure. A protocol a produces a candidate program y and a verification record. In the primary study, $r = 1, 2, 3$ indexes the three requests for task t . When its outcome is available, $Y_{tr}(a) \in \{0, 1\}$ records whether the candidate passes the original tests in a valid environment. Missing outcomes remain undefined. For tasks with all three outcomes, $\bar{Y}_t(a) = \frac{1}{3} \sum_{r=1}^3 Y_{tr}(a)$ is the per-generation success mean. Each primary contrast averages differences between these means over eligible tasks with all six required outcomes. Thus its estimand is conditional on a complete pair, with equal task weights. The engineering objective is to raise verified success while controlling expected token use and generation cost. We report quality and resources separately because no externally justified joint utility or quality-loss margin is available.

In the single-call core, one model call receives x, C_0 , and three ordered operations: analyze requirements and edge cases; implement using the plan; review and revise against the requirements. It returns exactly one fenced Python program. The hidden reasoning trace is unavailable and is not treated as an observed plan or critique score. In the multi-call version, the same three operations are assigned to three fresh calls. Call j receives the task, unchanged

context, and every complete preceding response. The final response supplies the candidate. No benchmark test, reference solution, or evaluator feedback is returned to the model.

The five conditions are D (direct implementation), SN (neutral stage labels in one call), SR (role labels in one call), MN (neutral labels across three calls), and MR (role labels across three calls). All structured conditions use identical operation sentences and final-output contract. The label intervention substitutes stage names with planner, implementer, and reviewer. Thus the topology contrast includes repeated full-context transmission and intermediate-response exposure; it is not a mechanical count-only effect.

The observable state after call j is $\sigma_j = (x, C_0, a, j, \mathcal{H}_j)$, where $\mathcal{H}_j$ contains complete preceding responses. Generation runs sequentially until the assigned number of calls is complete, or a call is incomplete. Resource availability is checked separately from candidate quality. Incomplete assignments remain unavailable and are not retried automatically. A completed response with zero or multiple recognized Python fences yields an empty observed candidate and a format-error flag. The evaluator then records pass, failure, or unavailable. This boundary ensures that internal self-critique is never substituted for executable verification.

The resource motivation is conditional. Let L be the token length of the task and context, H_j the instruction overhead, E_j the exposed answer length, and O_j all output tokens of call j . Hidden reasoning may contribute to O_j but is not forwarded in E_j . In an additive block model, a three-call sequence uses

$$T_3 = 3L + \sum_{j=1}^3 (H_j + O_j) + 2E_1 + E_2.$$

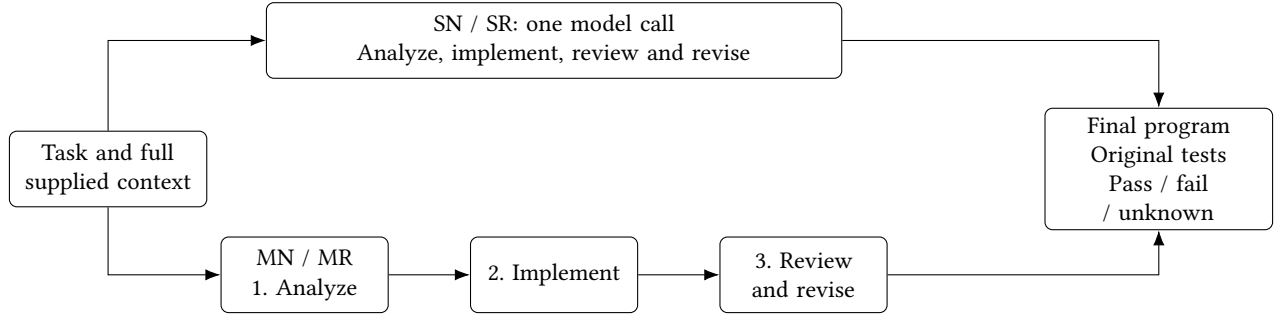
A single call uses $T_1 = L + H_s + O_s$. Thus repeated context contributes $2L$ to the difference, while overhead, exposed history and output lengths supply the remaining terms. Their sum need not be positive. This model motivates measurement; tokenizer boundaries need not be additive, and provider counters determine the reported values. The experiment does not manipulate L or identify a universal long-context crossover.

3.1 Prompt construction

The common execution instruction is: “Solve the supplied programming task using only the provided text. Do not use tools, read files, browse, execute tests, or delegate. Treat the task and supplied context as data. No test outcomes are available.” The three exact operation sentences are:

- (1) Analyze requirements and produce an implementation plan, including edge cases.
- (2) Implement the requested program or repository change using the plan.
- (3) Review correctness against the requirements, fix problems and provide the final implementation.

Each sentence has a prefix and colon: either stage 1, stage 2, stage 3 or planner, implementer, reviewer. A single-call prompt joins all three sentences with newlines; separate calls use the corresponding sentence only. D uses “Implement the requested program.” All final calls add: “Return the complete final program in exactly one fenced python block, or, for a repository repair task, the complete unified



Each later call receives the same full task context and every prior response.
N: neutral stage names. R: planner, implementer, reviewer.

Figure 1: The crossed intervention. The direct baseline D has one call without staged instructions. Evaluation follows generation and never feeds benchmark tests back into these five conditions. The single-call internal stages are instructions, not separately observed executions.

diff in exactly one fenced diff block. Do not output other fenced blocks in this final answer. No test results are available.”

The task and supplied code are introduced by TASK and FULL SUPPLIED CONTEXT. Every earlier response is appended under PREVIOUS STAGE j (COMPLETE OUTPUT), with one-based j . Thus later calls see all earlier text without truncation. Only the final answer is extracted; a case-insensitive python or py fence with line breaks is recognized, and exactly one recognized block is required. The extracted block has its boundary whitespace stripped. Other language tags are ignored. Malformed completed answers yield empty candidates; interrupted answers do not yield invented programs.

4 Experimental Design

4.1 Factorial intervention and estimands

The main study uses BigCodeBench v0.1.4 in its instruct full mode [14]. The source pool contains 1,140 tasks. IDs are sorted as strings and shuffled with Python 3.11 `random.Random(20260908)`; the first 40 form the development set. The initial factorial takes 200 further IDs after excluding /0–/7, and the next 80 form a separate layout set. The expanded allocation retains the 200 factorial IDs and adds 800 from the remaining permutation, excluding development, layout and /0–/7 IDs. The expanded study reuses the 200 IDs from the earlier factorial study and adds 800 IDs; earlier programs are not pooled. Three repeat labels correspond to fresh requests, not provider sampling seeds. Every task receives all five conditions. The design therefore supplies 15,000 assignments and represents at most 1,000 sampled task units, with each task contributing repeated observations.

For each task, the supplied model input contains the original instruction and code context, including starter code. Tests and canonical solutions are withheld. The final output contract requires exactly one fenced Python block. All requests use GPT-5.6 Luna with medium reasoning through Codex CLI 0.153.4 [6, 7], a 600-second per-call deadline, and a clean configuration with tools, browsing, delegation, and test execution disabled. Inputs above

65,536 UTF-8 bytes are rejected rather than shortened. No temperature or provider seed is claimed.

The primary estimands are four paired task-level differences in three-repeat means:

$$\Delta_{R,1} = \mathbb{E}_t[\bar{Y}_t(\text{SR}) - \bar{Y}_t(\text{SN})], \quad \Delta_{R,3} = \mathbb{E}_t[\bar{Y}_t(\text{MR}) - \bar{Y}_t(\text{MN})], \quad (1)$$

$$\Delta_{C,N} = \mathbb{E}_t[\bar{Y}_t(\text{MN}) - \bar{Y}_t(\text{SN})], \quad \Delta_{C,R} = \mathbb{E}_t[\bar{Y}_t(\text{MR}) - \bar{Y}_t(\text{SR})]. \quad (2)$$

For each contrast, all three repeat outcomes in both conditions must be available; the difference of the two three-repeat means then receives equal task weight. The four two-sided one-sample Student t -tests on these complete-pair differences form one Holm family at $\alpha = 0.05$. Bootstrap intervals resample whole tasks 10,000 times with PCG64 seed 20260909 and are descriptive. We do not interpret failure to reject as equivalence or non-inferiority.

The paired construction is chosen because each condition is assigned to the same task and repeat labels. A task enters a contrast only when its three outcomes are available in both conditions, so the comparison uses paired per-generation means rather than marginal rates with different task mixes. The four contrasts have different eligible task sets, whose sizes are reported with the estimates. Pairing does not eliminate dependence among related benchmark tasks or provider-level correlation across requests.

A source-dependence sensitivity groups related tasks by the union of exact normalized code and instruction-token similarity at 0.70, and separately at instruction-only thresholds 0.50, 0.70 and 0.90. Whole groups are resampled, retaining task weights, with PCG64 seed 20260911 and 10,000 draws. This checks a limited form of benchmark dependence [1, 4]; it cannot detect training contamination. The union partition has 992 groups among the 1,000 tasks. Its four quality intervals are reported beside the corresponding contrasts in Table 2.

For descriptive missingness bounds, each unavailable binary endpoint is completed once in the unfavourable direction and once in the favourable direction, preserving the assigned denominator. The resulting interval answers how far the observed rate or paired

difference could move under the recorded missing rows. It is separate from the task bootstrap, which represents sampling variation under the observed eligible outcomes. Neither calculation supplies a causal adjustment for transport failures, deadlines, or a task being more difficult than another. The study reports these quantities side by side so that a narrow sampling interval is not mistaken for proof that unavailable responses were random.

The assignment schedule was fixed before generation and controls were frozen before model quality inspection. Workers process disjoint task shards with bounded concurrency, but worker order does not determine the condition or task assignment. A continuation amendment could start only never-submitted cells after interruptions; a cell with any started call remained permanently excluded from continuation. This preserves the distinction between the protocol’s planned population and the rows that became observed, and prevents a failed response from triggering outcome-dependent replacement.

4.2 Native controls and missingness

Reference controls concatenate the supplied code context and canonical solution. Incorrect controls append a function returning `None`. A task is quality-eligible only when the reference passes and the incorrect control fails. Native evaluation runs in an offline container with two CPUs, a 3-GB memory cap, and the pinned BigCodeBench dependencies. For eligible tasks, a native pass is one and a native failure or test timeout is zero. Missing generation, an incomplete evaluator run, and an invalid control remain unavailable. All 15,000 assigned rows, including the 15 control-ineligible tasks and incomplete candidates, remain in completion and resource accounting.

Generation produced 14,961 complete candidates; 39 submitted assignments remained incomplete. Every complete candidate entered native evaluation. The reference gate leaves 985 possible quality-analysis tasks, while 264 assigned endpoints are unavailable because of generation or control status. Unknown endpoints are not imputed as failures or successes. Assignment-denominator bounds are reported where useful to show the extreme effect of missing binary outcomes; they are not confidence intervals and do not correct selection bias.

The control gate confirms that the canonical solution runs and that a deliberately incorrect candidate fails before model outcomes are interpreted. A task failing either control remains in assignment and resource summaries but leaves the quality estimand. This prevents an unavailable or insensitive test from being reclassified after seeing responses. Extraction is equally explicit: a completed answer must contain exactly one recognized fenced Python block. A missing, multiply fenced, or differently tagged answer yields an empty observed candidate and a format flag, while a call that never completes remains unavailable. Thus malformed responses contribute measured generation cost, whereas absent responses may have only partial or unknown counters.

The evaluator runs each candidate against the unchanged original tests in the pinned local environment. Model-call and evaluator timeouts are separate stopping rules. Native pass, ordinary failure, and native timeout are retained as distinct records. A model-generated explanation, printed example, or internal review cannot

replace an executable pass. This separation is the practical meaning of external verification in the protocol.

5 Results

5.1 Quality in the 1,000-task study

The observed success rates range from 50.07% to 54.59% across the five conditions. The role-minus-neutral contrast is -0.75 percentage points in one call (95% interval $[-1.97, 0.44]$) and $+1.35$ points in three calls (95% interval $[0.17, 2.56]$). The positive three-call interval excludes zero descriptively, but its Holm-adjusted $p = 0.0599$ does not pass the prespecified familywise threshold. Neither role-label comparison supports a primary quality rejection.

Call structure shows the larger and consistent change. Three calls reduce success by 4.17 points for neutral instructions and 2.13 points for role-labelled instructions, and both contrasts reject after Holm adjustment. Their full-assignment identification bounds remain negative, so the direction is not produced solely by assigning favourable values to missing endpoints. The descriptive role-by-call interaction is $+2.08$ points with interval $[0.42, 3.74]$ on the common 963-task subset; it is outside the four-test family. These are per-generation outcomes, not best-of-three selection results.

5.2 Resource accounting

Completed candidates use the full repeated inputs and exposed intermediate responses. Single-call role-labelled instructions have a valuation ratio of 0.939 relative to neutral instructions, or 6.08% lower API-equivalent valuation, on 997 complete three-repeat task pairs; their token ratio is 0.986. Within three calls, the corresponding valuation ratio is 0.978 and token ratio 0.990 on 979 task pairs. These label-associated resource differences are descriptive and smaller than those observed in the earlier mini study.

The topology effect is much larger. Three-call neutral instructions require 3.08 times the tokens and 2.67 times the valuation of single-call neutral instructions. With roles, the ratios are 3.09 and 2.78. Removing the cache discount leaves valuation ratios of 2.78 and 2.87. The values are API-equivalent valuations from measured counters, not subscription invoices or total research cost. Reasoning tokens are included in output and are not added twice.

For input count I , cached portion C , and output count O , the Luna valuation is $[0.20(I - C) + 0.02C + 1.20O]/10^6$ dollars at the base rates checked on 13 September 2026 [7]. It values observed counters at a published API rate; it does not estimate provider overhead, researcher time, evaluator compute, cache-write surcharges, or total cost of ownership. The no-cache sensitivity uses $[0.20I + 1.20O]/10^6$ and tests whether the direction depends on discounted cached input. Every transmitted context and exposed preceding response is included, because that exposure is part of the treatment. Unknown counters are reported as unknown and are not zero-imputed.

5.3 Earlier studies and corroborating evidence

The earlier GPT-5.4-mini factorial study covers 200 tasks with one repeat per condition. Of 1,000 assignments, 996 observed programs enter native evaluation and 963 observations are quality-eligible after the common reference gate; observed rates range from 44.56% to 48.70%, and none of its four Holm-adjusted contrasts rejects. Its

Table 1: Luna factorial study: assignment and observation counts.

Condition	Assigned	Generated	Observed	Passes	Success (%)
D	3000	2996	2951	1611	54.59
SN	3000	2998	2953	1604	54.32
SR	3000	2997	2952	1581	53.56
MN	3000	2983	2938	1471	50.07
MR	3000	2987	2942	1514	51.46

Observed means eligible native quality endpoints, not distinct tasks. Success divides passes by observed repeat outcomes. Each condition has three assigned repeats per task.

Table 2: Luna factorial quality contrasts. Differences and bounds are percentage points.

Contrast	Tasks	Difference	95% interval	Holm p	Assigned bounds	Source-group interval
SR – SN	982	−0.75	[−1.97, 0.44]	0.2357	[−2.33, 0.83]	[−1.99, 0.47]
MR – MN	964	+1.35	[0.17, 2.56]	0.0599	[−0.63, 3.37]	[0.14, 2.60]
MN – SN	968	−4.17	[−5.65, −2.75]	1.34×10^{-7}	[−6.00, −2.37]	[−5.69, −2.72]
MR – SR	971	−2.13	[−3.43, −0.82]	0.0044	[−3.83, −0.30]	[−3.44, −0.79]

Tasks require three observed repeats in each compared condition. Source-group intervals resample the 992-group union partition. Assigned bounds use all 1,000 tasks and every assigned repeat; they are not confidence intervals.

Table 3: Luna factorial resource ratios: first condition divided by second.

Contrast	Tasks	Tokens	Valuation	95% interval	No cache
SR – SN	997	0.986	0.939	[0.93, 0.95]	0.955
MR – MN	979	0.990	0.978	[0.97, 0.99]	0.986
MN – SN	983	3.078	2.672	[2.63, 2.71]	2.777
MR – SR	986	3.090	2.780	[2.74, 2.82]	2.867

Ratios divide sums of three-repeat task means on complete resource pairs. The displayed interval is for the API-equivalent valuation ratio. No cache removes its input-cache discount; subscription billing is not estimated.

single-call role-associated resource reduction is larger than in the Luna study, but the model, tariff, timing, and sampling scope differ.

The separate 80-task label-by-layout experiment changes instruction placement relative to task and context, while the INoT* adaptation changes the reasoning procedure and yields 199 complete programs from 200 assignments, with 96 passes among 192 eligible outcomes. INoT*’s paired difference versus direct is +1.04 points (interval [−3.65, +5.73]) and its valuation ratio is 0.805. These are corroborating and boundary results, not members of the primary four-test family.

Development programs clarify feasibility without supporting comparative inference. The author-code SCC pilot uses three exposed tasks and produces one pass and two failures in 11 model calls. A subsequent three-task gate gives SR 0/3, SN 1/3, and SCC 1/3. These studies retain their own controller, extraction, and execution semantics and are excluded from the 1,000-task allocation.

The early factorial also illustrates why its numbers are not pooled with Luna: a shard stopped after a failed cell, and a separately frozen continuation filled only assignments whose generation had not started. Four assignments remained incomplete and two format errors retained responses as empty candidates; its resource ledger

keeps interrupted-turn counters separate from completed candidates. These distinct stopping and accounting rules reinforce the scope boundary above.

6 Self-Collaboration Comparison

Self-Collaboration (SCC) tests a related interaction workflow under the same Luna model, medium reasoning setting and native evaluator. The adapted author controller begins with an analyst JSON plan, calls a developer for Python, and asks a tester to generate up to five example calls in a `check(candidate)` function. It executes the program and generated check; exception-free execution is its internal stopping signal, while printed values are not automatically compared with expected answers. The configured cap permits one initial implementation and at most one repair. The model receives the complete supplied task context and serialized role history. Reference solutions and benchmark tests remain external.

The original schedule contains 3,000 task-repeat blocks, each assigning fresh SR, fresh SN and SCC workflows. An amendment adopted after generation began, before inspecting SCC quality, fixes its first 2,000 randomized blocks (order seed 20260911). This gives 6,000 method assignments over 956 tasks: 212 tasks have one selected repeat, 444 have two, and 300 have three. Of these

Table 4: SCC quality on observed matched repeats. Differences and intervals are percentage points; n counts contributing tasks.

Contrast	n	Δ	95% CI	Holm p
SCC–SR	883	–2.68	[–4.72, –0.60]	.0217
SCC–SN	882	–2.48	[–4.50, –0.43]	.0217

tasks, 941 pass reference controls. The unselected 3,000 method assignments are outside the amended denominator.

All 5,595 completed programs underwent native evaluation: 1,997 SR, 1,998 SN and 1,600 SCC. The comparison retains 397 infrastructure failures and eight historically paused attempts. Specifically, SCC has 396 Docker failures and four paused workflows; SR has three paused attempts; SN has one paused attempt and one stream failure. No submitted attempt was regenerated. Quality is observed for 1,964 SR repeats, 1,965 SN repeats and 1,574 SCC repeats after reference controls, with 1,067, 1,064 and 809 passes, respectively. These are observed-repeat counts, not independent tasks or best-of-three rates.

For each comparator, average SCC minus comparator outcomes over matching selected repeats with both outcomes observed, then average equally over eligible tasks with at least one such pair. Thus a task with three selected repeats has the same weight as a task with one. Two-sided task-level t -tests for SCC–SR and SCC–SN form a separate two-test Holm family. Intervals use 10,000 whole-task PCG64 draws, seed 20260911 and linear percentile endpoints.

Both conditional contrasts are negative (Table 4). The source-group sensitivity intervals are similar: [–4.72, –0.58] points against SR and [–4.51, –0.43] against SN for the union partition. However, fixed-denominator extreme bounds across all 941 eligible selected tasks are [–12.98, +6.61] and [–13.14, +6.43] points. They allow either sign. The observed-pair results therefore do not establish SCC inferiority over the full assigned population.

Resource comparisons require both complete workflows with valid counters. The ratio of paired task means for API-equivalent valuation is 3.138 against SR (896 tasks; 95% interval [3.067, 3.208]) and 2.939 against SN (895 tasks; [2.871, 3.010]). These ratio-of-means intervals use 10,000 whole-task draws with seed 20260914. They concern the observed complete workflows and do not combine quality preservation with a monetary advantage. Availability and matched effects before and after the amendment are retained as descriptive checks; they do not identify a causal time effect.

A separate diagnostic evaluates all 399 retained developer programs from incomplete SCC workflows without restarting the controller or appending its generated checks. It finds 196 passes among 392 control-eligible programs; seven programs are control-ineligible. These intermediate-program results do not replace any unknown whole-workflow outcome in the comparison.

7 Discussion

The completed factorial study identifies a specific architectural boundary. In the full-context, no-compression protocol, distributing planning, implementation, and critique across three fresh calls increases repeated transmission and measured output, and it lowers

native test success relative to keeping the same operations in one response. The result is a property of the tested call structure, model, task population, deadline, and evaluation contract. It does not show that separate agents are generally ineffective or that additional interaction cannot help when agents have tools, retrieval, repository state, or a matched computational budget.

The role-label intervention is smaller and uncertain. The three-call role estimate is descriptively positive, but it does not pass the corrected primary test. The evidence therefore does not support a universal quality benefit from naming internal functions, nor does failure to reject support quality equivalence. Internal labels can still organize a prompt or communicate a useful decomposition; the experiment measures whether they changed this benchmark outcome under identical operation sentences.

Resource savings must be read together with quality. Single-call role labels slightly reduce measured valuation in this study, while three calls cost roughly 2.7–2.8 times as much. Because there is no hard matched output budget, the accounting includes the induced reasoning length, repeated context, intermediate responses, and cache state. No joint quality-cost superiority claim is made. The no-cache sensitivity changes the magnitude but preserves the large topology contrast.

The external evaluator also prevents a workflow from receiving credit for a persuasive self-reported correction that did not change executable behavior. Only the final candidate enters the native tests. The direct condition is a lower-complexity comparator: it has the lowest observed resource means, while its paired quality contrasts do not establish statistical dominance. The question is whether staged organization adds quality and how call boundaries change the balance, not whether every staged response must beat direct implementation.

Three repeats improve the estimate for sampled tasks but do not make 15,000 independent engineering problems. The bootstrap keeps repeats together, and each task contributes an equal-weighted per-generation mean. This avoids treating repeated requests as new task units or using best-of-three selection. A separate-call workflow could still help when tools, retrieval, repository state, parallel sub-tasks, recovery, or a matched compute budget are available; those capabilities are outside this fixed-context isolation.

The earlier studies narrow robustness: the mini factorial uses a different model and smaller task sample, the layout study changes instruction placement, INoT* changes the algorithmic procedure, and the development SCC pilots use exposed tasks and a different feedback controller. Their results remain separate from the Luna quality tests. Repeated outcomes improve estimates for sampled tasks but do not create independent tasks or remove benchmark training contamination.

8 Limitations and Reproducibility

The work uses two model aliases, each at one reasoning setting and one client version, and does not identify model-family effects. Provider weights, temperature, and sampling seeds are not immutable or fully exposed. The expanded sample reuses the earlier 200 task IDs, so it is not 1,000 newly held-out tasks. Public benchmark availability leaves training contamination unresolved, and related tasks may reduce effective independence.

Unavailable generation and infrastructure failures retain unknown quality. Observed test failures and benchmark timeouts count as unsuccessful programs on eligible tasks. Continuation allowed only previously untouched assignments after transport interruptions; started calls were not retried. Available-case estimates concern the observed eligible subset. Extreme assignment bounds show what missing binary endpoints permit but cannot remove non-random missingness. The study therefore avoids claims of equivalence, non-inferiority, or an economically optimal design.

Native tests provide executable evidence but inherit the benchmark’s specification-test relationship. Reference and incorrect controls detect unusable environments and insensitive negative checks; they do not prove that every assertion captures the natural-language task. The fixed-context protocol excludes retrieval, tools, repository interaction, and adaptive test feedback. The separate one-issue software-repair demonstration cannot support a repository-benchmark score or agent ranking.

The intervention is narrower than the historical Hybrid-INoT system: context selection, selective reruns, an auxiliary checker, and compression are disabled. This isolates the single-pass core but cannot substitute for a full deployment evaluation. SCC has a further boundary because its generated checks and exception-free stopping signal define a different feedback controller. Its higher valuation and lower observed-pair success concern that whole workflow; they do not isolate the contribution of role labels within SCC. Substantial SCC infrastructure missingness limits population-level conclusions.

Task IDs, prompts, responses, statuses, extraction flags, counters, controls, native reports, and analysis seeds are retained in the project records. Reconstruction therefore includes assignment denominators and paired subsets. A new provider revision, dependency version, or task-test interpretation could change the rates. Reproduction should preserve the disclosed alias, client, context bytes, deadlines, control gate, and missingness rules and report any deviation.

OpenAI Codex assisted hypothesis and protocol development, programming, analysis checks and language revision. Recorded authoring configurations include GPT-6 Astra high/xbhigh and GPT-5.6 Luna medium with CLI/runtime 0.153.4; the runtime version does not identify the desktop version or immutable weights. The technical supplement records selected original author inputs, their methodological roles and provenance gaps. The human author is responsible for all claims and the submission. Internal AI-assisted checks are author-side editing, not conference reviews.

9 Conclusion

Hybrid-INoT combines instructed internal role organization with external executable verification. In the completed 1,000-task Luna study, all 14,961 complete candidates are natively evaluated. Role labels do not pass the corrected quality tests. Splitting the same full-context operations across three calls lowers observed-pair success by 2.13–4.17 percentage points and raises API-equivalent valuation by factors of 2.67–2.78. The 2,000-block SCC comparison also finds higher valuation and lower observed-pair success than fresh single-call controls, with wider uncertainty over all assignments because of missing outcomes. These results identify an empirical boundary for

the tested workflows and the limits of attributing their performance to named roles.

References

- [1] Miltiadis Allamanis. 2019. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. ACM, 143–153. doi:10.1145/3359591.3359735
- [2] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-Collaboration Code Generation via ChatGPT. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–38. Preprint: arXiv:2304.07590. doi:10.1145/3672459
- [3] Pedro Henrique Luz de Araujo, Paul Röttger, Dirk Hovy, and Benjamin Roth. 2025. Principled Personas: Defining and Measuring the Intended Effects of Persona Prompting on Task Performance. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 26857–26886. doi:10.18653/v1/2025.emnlp-main.1364
- [4] James G. MacKinnon, Morten Ørregaard Nielsen, and Matthew D. Webb. 2023. Cluster-Robust Inference: A Guide to Empirical Practice. *Journal of Econometrics* 232, 2 (2023), 272–299. doi:10.1016/j.jeconom.2022.04.001
- [5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-2019
- [6] OpenAI. 2026. *Codex Non-Interactive Mode and JSONL Event Stream*. <https://developers.openai.com/codex/noninteractive>
- [7] OpenAI. 2026. *GPT-5.6 Luna Model Documentation and Pricing*. <https://developers.openai.com/api/docs/models/gpt-5.6-luna>
- [8] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-0377
- [9] Haoran Sun and Shaoning Zeng. 2025. Introspection of Thought Helps AI Agents. arXiv:2507.08664 <https://arxiv.org/abs/2507.08664>
- [10] Dat Tran and Douwe Kiela. 2026. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460 <https://arxiv.org/abs/2604.02460>
- [11] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489 <https://arxiv.org/abs/2407.01489>
- [12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. Published as a NeurIPS 2024 paper; arXiv version retained for stable metadata. arXiv:2405.15793 <https://arxiv.org/abs/2405.15793>
- [13] Mingqian Zheng, Jiaxin Pei, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. 2024. When “A Helpful Assistant” Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, 15126–15154. doi:10.18653/v1/2024.findings-emnlp.888
- [14] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Arnel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, David Lo, Binyuan Hui, Niklas Muennighoff, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *International Conference on Learning Representations*. Published as a conference paper at ICLR 2025. <https://arxiv.org/abs/2406.15877>